第7章 第10节 需求分析与流程

- 来源: https://class.imooc.com/lesson/5281#mid=104939
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第7周 MCP：模型上下文协议
- 模块: MCP通信协议与实战
- 课节: 图文 1-3 需求分析与流程

1-3 需求分析与流程

第7章 第10节 需求分析与流程

> [!TIP] 阅读指南 本节不再讲概念，而是从零开始实现一个游戏装备计算助手。先做需求分析和交互流程设计，搞清楚要做什么、怎么运转。

10.1 需求分析：从玩家痛点出发

项目目标

在 RPG 或 MOBA 类游戏中，玩家常面临复杂的装备属性抉择：这把新武器比旧的强多少？换了这件装备，总伤害能提升几成？传统的做法是查攻略或拉 Excel 表格，既繁琐又不直观。

我们的目标是打造一个「游戏助手」，让玩家只需通过自然语言提问，即可获得精准的数值计算与对比建议。

理想的交互体验

```
你："这把「破晓之剑」比我现在的武器强多少？"

AI："让我帮你计算一下……
     破晓之剑：基础攻击 674，相比当前武器提升了 32.2%。
     算上暴击加成，总伤害预计提升约 40%。建议更换！"
```

这就是我们要做的事情。

为什么需要 MCP

我们可能会疑惑，让AI学会计算游戏数值就好了，为什么要用MCP?

让我们对比两种实现方式。

方案A：纯提示词方案（不用MCP）

```
你：帮我算一下破晓之剑的总伤害

AI：要计算总伤害，需要基础攻击力、暴击率、
    暴击伤害等数据。请问这把武器的具体
    属性是……（开始问一堆问题）
```

或者，AI干脆会给你一个公式，让你自己去计算。

问题在于：

```
[×] AI不知道装备数据，你得手动输入
[×] 每次查询都要重复说明装备属性
[×] AI没有"记忆"，无法对比多件装备
```

方案B：MCP方案（我们要做的）

```
你：帮我算一下破晓之剑的总伤害

AI：【调用 get_equipment 工具查询装备数据】
   【调用 calculate_damage 工具计算伤害】

    破晓之剑总伤害：2,847
    （基础攻击674 × 暴击加成2.85 × 其他加成1.48）
```

这就是MCP的价值: 它让AI从"会说"变成"会做"。更重要的是，你可以把这个MCP发布出去，让其他的AI应用也可以使用你的MCP Server。

我们要实现什么

基于上面的分析，我们的目标很明确：

项目定位：游戏装备计算助手

- 核心功能：装备查询、伤害计算、对比分析、升级规划

核心功能

- 交互方式：自然语言对话

交互方式

- 技术方案：AI Client + MCP Server

技术方案

具体来说，玩家可以：

1. 查询装备属性

```
"破晓之剑有什么属性？"
→ 通过Resources查询装备数据，返回详细属性

1
2
```

2. 计算战斗数值

```
"我用破晓之剑打史莱姆能造成多少伤害？"
→ 调用伤害计算工具，返回准确结果

1
2
```

3. 对比装备性能

```
"破晓之剑和无锋重剑哪个更强？"
→ 调用对比分析工具，给出建议

1
2
```

完整交互流程：从 LLM 决策到最终输出

现在让我展示一个完整的交互流程图，看看用户问一个问题后，系统是如何一步步地处理并生成最终答案的：

完整流程解读

1. 用户提问阶段

- 用户输入问题
- AI Client 接收并处理输入

2. LLM 决策阶段

- Client 将问题发送给 LLM（通义千问）
- 关键：Client 会携带 MCP Server 提供的所有能力列表（Resources/Tools/Prompts）
- LLM 分析问题，自主决定需要调用哪种能力： Resources：查询装备数据 Tools：执行计算或对比 Prompts：获取分析模板 None：不需要 MCP 能力直接回答

3. MCP Server 执行阶段

- Client 根据 LLM 的决策，调用对应的 MCP 能力
- MCP Server 执行具体操作： Resources 模块：读取装备数据，返回格式化卡片 Tools 模块：执行复杂计算（如对比、伤害计算） Prompts 模块：返回预设的 Prompt 模板
- 所有模块都从 equipments.json 读取数据

```
equipments.json
```

4. LLM 生成阶段

- MCP Server 将结果返回给 Client
- Client 将结果作为上下文发送给 LLM
- LLM 基于 MCP 返回的数据生成答案

5. 循环判断

- LLM 判断当前信息是否足够回答问题

- 需要更多信息：返回决策阶段，继续调用 MCP 能力（虚线箭头）

需要更多信息

- 信息已足够：进入输出阶段

信息已足够

6. 最终输出

- LLM 生成完整的自然语言回答
- 返回给用户

核心要点

```
[√] LLM 全程主导 - 自主决定调用哪些 MCP 能力
[√] 自动循环 - 可多次调用，直到信息足够
[√] 最终由 LLM 生成 - 保证输出自然流畅
[√] MCP Server 只负责执行 - 不做决策，只提供数据/计算
```

不同场景下的调用次数示例

场景 1：调用 1 次 MCP

```
用户输入：「请查询装备"破晓之剑"的属性」
    ↓
LLM 分析：需要读取装备数据
    ↓
LLM 决定：调用 Resources
    ↓
调用 MCP：read_resource(weapon_破晓之剑) ← 第 1 次
    ↓
MCP 返回：装备卡片数据（类型、攻击、暴击等）
    ↓
LLM 判断：信息已足够
    ↓
生成答案：输出该装备的属性说明
```

总计：1 次 MCP 调用

场景 2：调用 3 次 MCP

```
用户问："帮我详细分析破晓之剑，并推荐搭配的护甲"
    ↓
LLM 分析：需要分析模板
    ↓
LLM 决定：调用 Prompts
    ↓
调用 MCP：get_prompt(equipment_analysis) ← 第 1 次
    ↓
MCP 返回：分析模板（属性解读、适用场景、优缺点、搭配建议）
    ↓
LLM 判断：需要装备详细数据
    ↓
LLM 决定：调用 Resources
    ↓
调用 MCP：read_resource(weapon_破晓之剑) ← 第 2 次
    ↓
MCP 返回：破晓之剑完整属性
    ↓
LLM 判断：需要护甲数据用于搭配建议
    ↓
LLM 决定：调用 Resources
    ↓
调用 MCP：read_resource(armor_龙鳞甲) ← 第 3 次
    ↓
MCP 返回：龙鳞甲属性
    ↓
LLM 判断：信息已足够
    ↓
生成答案：完整的分析报告 + 搭配建议
```

总计：3 次 MCP 调用

场景 3：调用 5 次 MCP

```
用户问："对比所有武器，给我推荐最适合暴击流的装备组合"
    ↓
LLM 分析：需要获取所有武器数据
    ↓
LLM 决定：调用 Resources
    ↓
调用 MCP：read_resource(weapon_破晓之剑) ← 第 1 次
调用 MCP：read_resource(weapon_无锋重剑) ← 第 2 次
调用 MCP：read_resource(weapon_疾风短刃) ← 第 3 次
    ↓
MCP 返回：3 件武器的完整属性
    ↓
LLM 判断：需要计算暴击收益
    ↓
LLM 决定：调用 Tools
    ↓
调用 MCP：calculate_damage(疾风短刃) ← 第 4 次
    ↓
MCP 返回：疾风短刃伤害详情（暴击率 75%）
    ↓
LLM 判断：需要护甲搭配
    ↓
LLM 决定：调用 Resources
    ↓
调用 MCP：read_resource(armor_暗影斗篷) ← 第 5 次
    ↓
MCP 返回：暗影斗篷属性
    ↓
LLM 判断：信息已足够
    ↓
生成答案："暴击流推荐：疾风短刃（75% 暴击率）+ 暗影斗篷，
          总暴击收益最高..."
```

总计：5 次 MCP 调用

三种 MCP 能力的对比

能力	LLM 何时选择	适用场景
Resources	需要查询装备的详细信息	简单数据查询
Tools	需要计算、对比、复杂逻辑	复杂计算
Prompts	需要结构化分析模板	深度分析

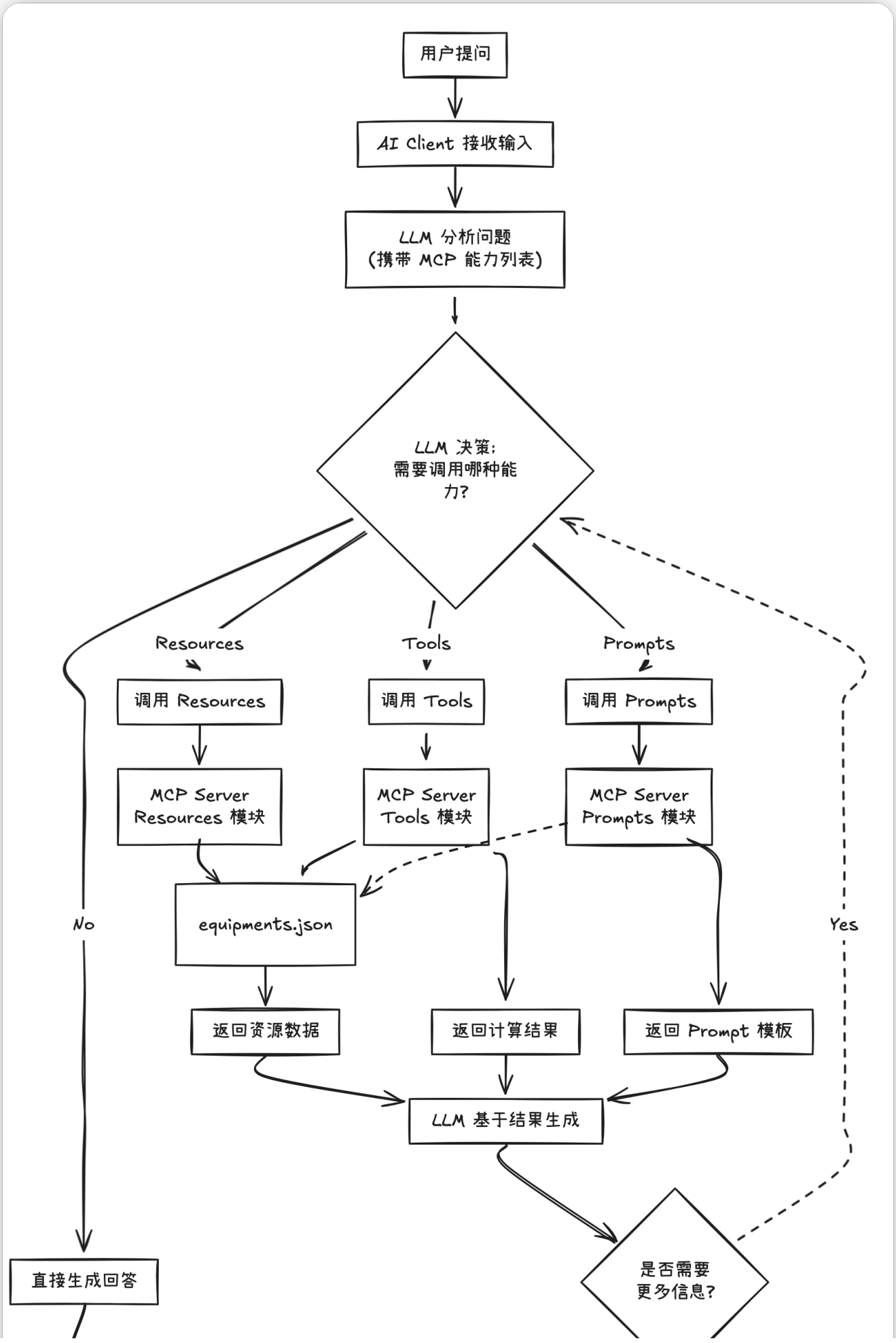
10.2 下一节预告

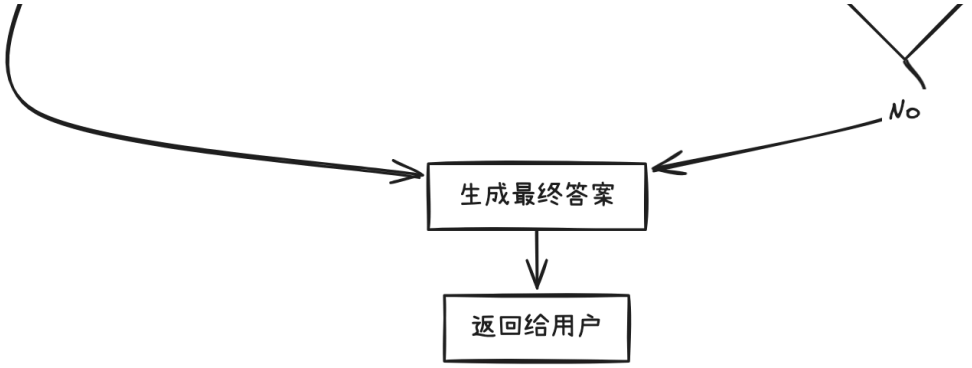
需求分析和交互流程已经清楚了。接下来进入技术选型与实现环节——为什么 Server 用 Node.js、Client 用 Python？项目的整体架构和目录结构是怎样的？下一节将给出答案。

10.3 ■ 学点英语

中文	English	音标	说明
交互流程设计	Interaction Flow Design	/ˌɪntərˈækʃn floʊ dɪˈzaɪn/	设计用户、AI应用和MCP Server之间的多步交互流程
渐进式决策	Progressive Decision	/prəˈgresɪv dɪˈsɪʒn/	LLM在调用过程中持续观察结果，动态决定后续步骤
参数引导	Parameter Guidance	/pəˈræmɪtər ˈgaɪdns/	Prompt在参数缺失时主动询问用户补充的交互机制
资源调用计数	Resource Call Count	/ˈriːsɔːrs kɔːl kaʊnt/	完成一个用户需求所需的MCP调用次数统计

图片


用户提问
AI Client 接收输入
LLM 分析问题
(携带 MCP 能力列表)
LLM 决策:
需要调用哪种能
力?
Resources
Tools
Prompts
调用 Resources
调用 Tools
调用 Prompts
MCP Server
Resources 模块
MCP Server
Tools 模块
MCP Server
Prompts 模块
equipments.json
No
Yes
返回资源数据
返回计算结果
返回 Prompt 模板
LLM 基于结果生成
直接生成回答
是否需要
更多信息?



生成最终答案

返回给用户